

Que debemos hacer:

1. Un sistema que tenga:
 - Un **Gateway** (puerta principal)
 - Un **Auth** (login con usuario y contraseña)
 - Un **Catalog** (servicio simple)
 - Un **Order** (servicio simple)
2. Que el login genere un **token (JWT)**.
3. Que si no hay token → dé **401 (no autorizado)**.
4. Que si hay token válido → deje pasar.

C1RM-1 — Catalog Service

OBJETIVO:

Crear el microservicio Catalog (8082) con endpoint de prueba y health.

TAREAS (JAVA):

- 1) Crear proyecto Spring Boot: services/catalog-service
- 2) Configurar server.port=8082 en application.yml
- 3) Crear controller: PingController con GET /ping que responda "catalog ok"
- 4) Habilitar actuator health (/actuator/health)

C1RM-2 — Order Service

OBJETIVO:

Crear el microservicio Order (8083) con endpoint /ping y health.

TAREAS (JAVA):

- 1) Crear proyecto Spring Boot: services/order-service
- 2) Configurar server.port=8083
- 3) Crear controller: PingController con GET /ping que responda "order ok"
- 4) Habilitar actuator health

C1RM-3 — API Gateway Rutas

OBJETIVO:

Configurar el API Gateway (8080) para enrutar /auth, /catalog, /orders.

TAREAS (JAVA + CONFIG):

- 1) Crear proyecto: services/api-gateway
- 2) Configurar server.port=8080
- 3) Configurar rutas:
 - /auth/** -> http://localhost:8081
 - /catalog/** -> http://localhost:8082 con StripPrefix=1
 - /orders/** -> http://localhost:8083 con StripPrefix=1
- 4) Crear un controller Java simple en gateway:
 - GET /gateway/ping -> "gateway ok"(para evidencia y para que esta persona tenga Java seguro)

C1RM-4 — Auth Base

OBJETIVO:

Crear la base del Auth Service (8081) con JPA para usuarios (tabla creada por código).

TAREAS (JAVA):

- 1) Crear proyecto: services/auth-service
- 2) Configurar conexión PostgreSQL a authdb (application.yml)
- 3) Crear:
 - Role enum (ADMIN, CUSTOMER)
 - UserEntity (email, password, role)
 - UserRepository (findByEmail, existsByEmail)
- 4) Configurar JPA: ddl-auto=update para crear tablas automáticamente.

C1RM-5 — Auth Register + BCrypt

OBJETIVO:

Implementar registro de usuario con contraseña encriptada (BCrypt).

TAREAS (JAVA):

- 1) Crear DTO RegisterRequest (email, password, role)
- 2) Crear PasswordConfig con PasswordEncoder BCrypt
- 3) Implementar AuthService.register():
 - validar email único
 - guardar password encriptada
 - role por defecto CUSTOMER si no llega
- 4) Endpoint: POST /auth/register en AuthController

C1RM-6 — Auth Login + JWT

OBJETIVO:

Implementar login que devuelva JWT con expiración y rol en el token.

TAREAS (JAVA):

- 1) Crear DTO LoginRequest (email, password)
- 2) Crear DTO TokenResponse (token, type)
- 3) Crear JwtService para generar JWT:
 - subject=email
 - claim role
 - exp=15 min
- 4) Implementar AuthService.login():
 - si credenciales inválidas -> 401
 - si ok -> devolver TokenResponse
- 5) Endpoint: POST /auth/login

C1RM-7 — Gateway Security (401/403)

OBJETIVO:

Proteger /orders/** en el Gateway con validación JWT y roles.

TAREAS (JAVA):

- 1) En api-gateway crear:
 - RouteValidator (define qué rutas se protegen: /orders/**)
 - JwtAuthFilter (valida Authorization: Bearer)
- 2) Comportamiento:
 - Sin token -> 401
 - Token inválido/expirado -> 401
 - Token válido pero role != ADMIN -> 403 (Forbidden)
 - Token válido con role ADMIN -> permitir paso
- 3) Asegurar que gateway use la misma secret del auth-service.

C1RM-8 — “Integración y Calidad”

OBJETIVO:

Mejorar la calidad del Sprint 1 agregando manejo de errores en Auth y una respuesta estándar en ping (JSON),
además de cerrar pruebas finales de integración.

TAREAS (JAVA):

1) En auth-service crear GlobalExceptionHandler (RestControllerAdvice):

- capturar ResponseStatusException
- responder JSON: { "status": 401, "error": "..." }

2) En catalog-service y order-service ajustar PingController para responder JSON (no solo string):

- ejemplo: { "service": "catalog", "status": "ok" }

(esto es código Java real y demuestra calidad)

3) Verificar integración final:

- register -> login -> token -> orders/ping (401/403/200)
- health de los 4 servicios